

Supplementary Material: A Differentiable Atari VCS: A Complex, Fully Known Ground Truth for Explainable AI

Anonymous submission

This supplement gives the full proofs of the two theorems stated in the main paper, together with the implementation-effort plot referenced in the Results. It is self-contained: we first restate the soft/hard formulation, then prove the forward-equivalence and temperature-limit results, and finally show the effort timeline.

1 Setup and Notation

The differentiable emulator runs one of two transition maps over the full VCS state $x = (\text{registers}, \text{RAM}, \text{RIOT}, \text{TIA}, \text{frame buffer})$: the bit-exact *hard* map Φ_H and the differentiable *soft* map Φ_S . In both, the handler that runs is the one for the opcode byte at the program counter pc . The handlers are assembled from four primitives, which we restate here so that the proofs are self-contained.

The first primitive is the memory read. A read of a memory tensor $r \in \mathbb{R}^M$ of size M (the ROM or the RAM) at an integer address $a \in \{0, \dots, M-1\}$ is written as a dot product against the one-hot address indicator $\mathbf{1}_a$,

$$\text{peek}(r, a) = \mathbf{1}_a^\top r = \sum_i \mathbb{I}[i = a] r_i = r_a, \quad (1)$$

so the forward value is the array element r_a , while the gradient with respect to r is the one-hot vector $\mathbf{1}_a$.

The second primitive is opcode dispatch. With a score vector (logits) $\ell \in \mathbb{R}^K$ over the $K = 256$ opcodes, a temperature $T > 0$, and the softmax weights $w = \text{softmax}(\ell/T)$, the relaxed dispatch over the candidate handler-output rows V is the convex combination

$$\text{select}(\ell, V; T) = w^\top V, \quad w_k = \frac{e^{\ell_k/T}}{\sum_j e^{\ell_j/T}}. \quad (2)$$

The third primitive is the conditional branch. For a relaxed status flag $f \in [0, 1]$ written as a logit $z = s(2f - 1)$, where the sign s selects branch-when-set or branch-when-clear, and a sharpness α , the gate and the relaxed program counter are

$$g = \sigma(\alpha z), \quad \text{pc}_{\text{soft}} = (1 - g) \text{pc}_{\bar{b}} + g \text{pc}_b = \text{pc}_{\bar{b}} + g \delta, \quad (3)$$

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

where $\text{pc}_{\bar{b}}$ is the fall-through (not-taken) counter and $\text{pc}_b = \text{pc}_{\bar{b}} + \delta$ is the taken counter for the signed branch offset δ .

The fourth primitive is the straight-through estimator. For any soft value that has a matching hard value it returns

$$\text{STE}(\text{soft}, \text{hard}) = \text{soft} + \text{sg}(\text{hard} - \text{soft}), \quad (4)$$

where $\text{sg}(\cdot)$ is the stop-gradient operator, so the forward value equals *hard* while the backward pass uses the gradient of *soft*.

In the executed soft step, dispatch uses the saturated (hard) form of (2) and the branch returns $\text{STE}(\text{pc}_{\text{soft}}, \text{pc}_{\text{hard}})$; the relaxed forms (2) and (3) *without* the straight-through correction define the “fully relaxed” variant analysed in Theorem 2. The temperature-limit result needs one non-degeneracy assumption, identical to the one in the main paper.

Assumption 1 (Decision margins). *Along the executed trajectory of length N , every discrete decision has a strictly positive margin. For opcode dispatch at step t with logits $\ell^{(t)}$ and winner k_t^* , the logit gap $\Delta_t = \ell_{k_t^*}^{(t)} - \max_{k \neq k_t^*} \ell_k^{(t)}$ is positive; for each conditional branch with flag logit z_t , the margin $m_t = |z_t|$ is positive. Let $\Delta_{\min} = \min_t \Delta_t > 0$ and $m_{\min} = \min_t m_t > 0$.*

2 Proofs

Theorem 1 (Exact forward equivalence). *For every reachable state x and every finite sharpness α and temperature T , the soft and hard one-step maps are equal, $\Phi_S(x) = \Phi_H(x)$, with identical float32 representations. Consequently, over a trajectory of N steps, $x_t^S = x_t^H$ for all $t \leq N$. The modes differ only in the Jacobian $\partial \Phi_S / \partial x$, which is defined and generically nonzero where $\partial \Phi_H / \partial x$ is zero or undefined.*

Proof. We first prove the one-step equality $\Phi_S(x) = \Phi_H(x)$ for an arbitrary reachable x , and then extend it to a trajectory of length N by induction on the step index.

One-step equality. Fix a reachable x . In both modes the handler that executes is selected by the decoded opcode byte through the same hard switch (the softmax form (2) is not evaluated on the forward path), so both modes run the same handler on the same input. It therefore suffices to show that each primitive the handler uses returns the same forward value in both modes. A memory read at the integer address a uses an exact one-hot indicator, so

$$\text{peek}_S(r, a) = \mathbf{1}_a^\top r = \sum_i \mathbb{I}[i = a] r_i = r_a = \text{peek}_H(r, a), \quad (5)$$

the value a hard array read returns. Register, status-flag, binary-coded-decimal, and timer updates use the same integer and round arithmetic in both modes (soft mode merely keeps the results in float32). A conditional branch returns the straight-through value (4); since sg is the identity in the forward direction,

$$\text{STE}(\text{pc}_{\text{soft}}, \text{pc}_{\text{hard}}) = \text{pc}_{\text{soft}} + \text{sg}(\text{pc}_{\text{hard}} - \text{pc}_{\text{soft}}) = \text{pc}_{\text{hard}} \quad (6)$$

for every α and T . Every component of the handler output thus equals its hard counterpart, and as the two handlers act on the same input,

$$\Phi_S(x) = \Phi_H(x). \quad (7)$$

These equalities hold in float32, not merely over the reals: IEEE-754 single precision has a 24-bit significand and so represents every integer in $[-2^{24}, 2^{24}]$ exactly (IEEE 2019; Goldberg 1991), and every VCS quantity lies in this range (data bytes ≤ 255 , 13-bit addresses, per-frame cycle counts of a few tens of thousands), so (5)–(7) are exact.

Induction on the trajectory. Let the two runs share the initial state and evolve by $x_{t+1}^\bullet = \Phi_\bullet(x_t^\bullet)$. We show $x_t^S = x_t^H$ for all $0 \leq t \leq N$.

Base case ($t = 0$): $x_0^S = x_0^H$ by the shared initial state.

Inductive step: assume $x_t^S = x_t^H =: x$ for some $t < N$. Applying the one-step equality (7) to x ,

$$x_{t+1}^S = \Phi_S(x_t^S) = \Phi_S(x) = \Phi_H(x) = \Phi_H(x_t^H) = x_{t+1}^H, \quad (8)$$

where the outer equalities use the inductive hypothesis and the middle one is (7). By induction $x_t^S = x_t^H$ for all $t \leq N$.

Gradients. The stop-gradient in (4) alters only the reverse-mode rule: the returned counter carries the derivative of $\text{pc}_{\text{soft}} = \text{pc}_{\bar{b}} + g \delta$, namely $\alpha g(1 - g) \delta$, which is generically nonzero where the hard branch—a step function of the flag—has zero or undefined derivative; likewise the read (1) has Jacobian $\mathbf{1}_a$ with respect to r . The forward pass is unchanged while gradients become available. $\square$

Theorem 2 (Temperature-limit bound). *Consider the fully relaxed emulator that uses the softmax dispatch (2) and sigmoid branch (3) without the straight-through correction. Under Assumption 1, its one-step map converges to the hard map as $T \rightarrow 0$ and $\alpha \rightarrow \infty$, with per-step deviation*

$$\|\Phi_S^T(x) - \Phi_H(x)\|_\infty \leq C[(K-1)e^{-\Delta_{\min}/T} + e^{-\alpha m_{\min}}], \quad (9)$$

where C bounds the value range (bytes ≤ 255 , branch offsets ≤ 256). Over N steps, with Lipschitz constant $L \geq 1$ for error propagation, the trajectory deviation is at most $\frac{L^N - 1}{L - 1} C[(K-1)e^{-\Delta_{\min}/T} + e^{-\alpha m_{\min}}] = \mathcal{O}(e^{-c/T})$ with $c = \Delta_{\min}$.

Proof. We bound the deviation of the relaxed one-step map from the hard one and then propagate it along the trajectory. Throughout, C bounds the range of any state value (bytes ≤ 255 , branch offsets ≤ 256).

Dispatch term. At a step with logits ℓ and winner k^* the gap is $\Delta \geq \Delta_{\min} > 0$ by Assumption 1, so for any loser

$k \neq k^*$,

$$w_k = \frac{e^{\ell_k/T}}{\sum_j e^{\ell_j/T}} \leq \frac{e^{\ell_k/T}}{e^{\ell_{k^*}/T}} = e^{(\ell_k - \ell_{k^*})/T} \leq e^{-\Delta_{\min}/T}, \quad (10)$$

so the non-winning mass is $1 - w_{k^*} = \sum_{k \neq k^*} w_k \leq (K-1)e^{-\Delta_{\min}/T}$. As the dispatch output is the convex combination $\sum_k w_k V_k$,

$$\left\| \sum_k w_k V_k - V_{k^*} \right\|_\infty \leq (1 - w_{k^*}) \max_k \|V_k - V_{k^*}\|_\infty \leq (K-1) e^{-\Delta_{\min}/T} \quad (11)$$

Branch term. For a margin $m = |z| \geq m_{\min}$ the gate error is

$$|\sigma(\alpha z) - \llbracket z > 0 \rrbracket| = \sigma(-\alpha m) \leq e^{-\alpha m_{\min}}, \quad (12)$$

so the blended counter $\text{pc}_{\bar{b}} + g \delta$ differs from the hard target $\text{pc}_{\bar{b}} + \llbracket z > 0 \rrbracket \delta$ by

$$|(g - \llbracket z > 0 \rrbracket) \delta| \leq |\delta| e^{-\alpha m_{\min}} \leq C e^{-\alpha m_{\min}}. \quad (13)$$

Per-step bound. Adding (11) and (13) bounds the one-step deviation: for every reachable x ,

$$\delta_{\text{step}} := \|\Phi_S^T(x) - \Phi_H(x)\|_\infty \leq C[(K-1)e^{-\Delta_{\min}/T} + e^{-\alpha m_{\min}}], \quad (14)$$

which is (9).

Propagation. Let $L \geq 1$ be a Lipschitz constant of Φ_H in $\|\cdot\|_\infty$ and $e_t = \|x_t^{T,S} - x_t^H\|_\infty$ the trajectory error after t steps, with $e_0 = 0$. Each step adds at most δ_{step} to the hard map applied to the accumulated error,

$$e_{t+1} \leq L e_t + \delta_{\text{step}}, \quad (15)$$

and unrolling from $e_0 = 0$ gives

$$e_N \leq \delta_{\text{step}} \sum_{i=0}^{N-1} L^i = \delta_{\text{step}} \frac{L^N - 1}{L - 1}. \quad (16)$$

As $T \rightarrow 0$ and $\alpha \rightarrow \infty$, $\delta_{\text{step}} = \mathcal{O}(e^{-\Delta_{\min}/T}) \rightarrow 0$, so the relaxed trajectory converges to the hard one, $\Phi_S^T \rightarrow \Phi_H$. $\square$

3 Effect of the Relaxation Parameters on the Gradient

Theorems 1 and 2 say the hard limit ($\alpha \rightarrow \infty$ for the sigmoid branch, $T \rightarrow 0$ for the softmax select) is exact in value but degenerate in gradient. Figure 1 shows this directly on jutari as the relaxation values are swept. We render a single player cannon whose horizontal position is produced by the soft primitives—a sigmoid-gated blend of a left and a right placement (`soft_branch`, sharpness α , top row) and a softmax mixture over candidate columns (`soft_select`, temperature T , bottom row)—and plot the screen-space directional-derivative saliency $\partial(\text{screen})/\partial(\text{control})$, that is, how each pixel’s value changes as the scalar control input increases. The colour encodes the *sign* of that derivative: red where a pixel brightens as the control increases (the sprite arriving) and blue where it darkens (the sprite leaving); the colour intensity is the magnitude. The two rows are on *individual* colour scales: the branch and select gradients are taken with

respect to different controls and differ in absolute magnitude by more than an order of magnitude (peak 0.90 vs. 0.05 here), so each row carries its own colour bar in absolute units, and the per-panel number is the peak relative to that row’s maximum. A pixel with no sensitivity maps to the same neutral light grey in every panel—the faint cannon outline—so the zero level reads identically in both rows.

For the branch the position is $pc = (1 - g) pc_L + g pc_R$ with $g = \sigma(\alpha z)$, so its derivative with respect to the control z is $\alpha \sigma(\alpha z) (1 - \sigma(\alpha z)) (pc_R - pc_L)$: a dipole, blue on the placement being vacated and red on the one being entered. The magnitude $\alpha \sigma(\alpha z) (1 - \sigma(\alpha z))$ is *non-monotonic* in α at the fixed operating point $z=0.25$. A small α gives a shallow gate and hence a small slope; a large α saturates the gate ($\sigma(\alpha z) \rightarrow 1$, so $\sigma(1 - \sigma) \rightarrow 0$). The maximum is the intermediate α for which αz falls in the sigmoid’s steep band, near $\alpha=6$ here— $\alpha=2, 6, 20$ give peak magnitudes 0.53, 1.00, 0.15, and at $\alpha=20$ the gate has essentially switched and the gradient has nearly vanished (the hard-branch limit). Figure 2 plots this gate and its gradient directly: at $\alpha=20$ the gate has saturated and dg/dz is essentially zero away from the switch (too sharp: no usable gradient), while at $\alpha=2$ the gate is so shallow that the gradient is weak and barely varies with z (too soft: the gradient does not discriminate); the intermediate $\alpha=6$ gives the largest, most localised gradient at the operating point.

For the select the rendered cannon is the softmax-weighted sum $\sum_k w_k$ (cannon at column k) with $w = \text{softmax}(\ell/T)$. At a high temperature the weights spread across several neighbouring candidate columns, so the render is a superposition of the cannon at those columns—the several faint copies seen at $T=2.0$ —and the gradient is spread over them. As T falls the weights concentrate: at $T=0.5$ only a few copies remain, and at $T=0.1$ the mixture is effectively one-hot, a single sharp cannon whose chosen column no longer moves under a small change of the control, so the gradient collapses to zero between candidate boundaries (peak magnitudes 1.00, 0.51, 0.00 for $T=2.0, 0.5, 0.1$). Figure 3 makes this concrete in pixel space: sampling the cannon’s column 100 times from $w = \text{softmax}(\ell/T)$ and averaging the rendered sprites gives the expected screen occupancy—a single sharp cannon at low T ($N_{\text{eff}} \rightarrow 1$) that spreads into several graded copies as T rises, the very superposition the gradient is then spread across.

In every panel the forward render is the exact hard one (Theorem 1); only the gradient changes. It is most informative at intermediate relaxation and decays to zero as the relaxation is sharpened toward the hard limit (Theorem 2). This is a qualitative study of how the parameter *values* act on the gradient, run on jutari; the values need not be tuned for the bit-exact forward pass. Animated sweeps of both knobs—the gate and its gradient as α varies, and the weight distribution as T varies, each frame on its own scale—are provided as `fig_alpha_anim.gif` and `fig_temp_anim.gif` in the supplementary media.

4 Implementation Effort

Figure 4 is the evidence behind the effort estimate in the main paper. Because every step of the project was committed, the

commit timestamps record when work actually happened. To turn them into an active-time figure we group the commits into sessions, treating a gap of more than three hours between consecutive commits as a session boundary. Three hours is well above the in-session rhythm—the median gap between consecutive commits is about 13 minutes—and well below the multi-hour and overnight gaps that separate genuine work periods, so the partition is insensitive to the exact threshold: the active total is 106 hours at a two-hour cutoff and 162 hours at a four-hour cutoff, bracketing the 137 hours obtained at three hours.

With the three-hour boundary the 373 commits fall into 52 sessions whose durations sum to 136.8 hours, i.e. about 5.7 eight-hour working days, even though they are spread over a 29-day calendar window. The remaining $\sim 80\%$ of the calendar span is idle and excluded; it consists mostly of stretches during which the lead programmer was travelling and without reliable internet access, which appear in the plot as the long flat segments. This active total—not the calendar span—is what we compare against the scale of the reference codebase in the main paper.

$\partial(\text{screen})/\partial(\text{control})$ vs. relaxation strength: largest at moderate relaxation, vanishing toward the hard limit ($\alpha \rightarrow \infty$, $T \rightarrow 0$)

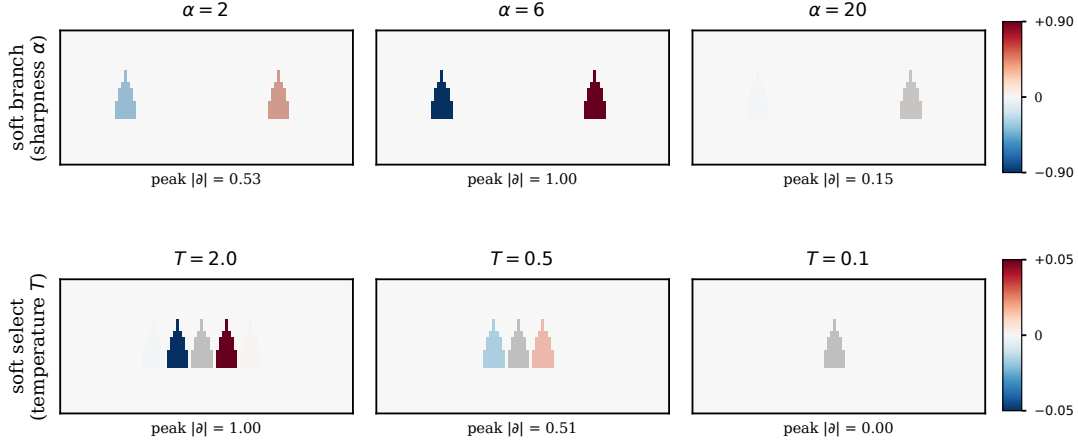


Figure 1: Screen-space gradient $\partial(\text{screen})/\partial(\text{control})$ on jutari as the relaxation strength is swept. **Top:** sigmoid branch (soft_branch), increasing sharpness α ; the gradient is a dipole between the left and right placements. **Bottom:** softmax select (soft_select), decreasing temperature T ; a high T renders the cannon as several weighted copies. Colour gives the sign of the per-pixel derivative (red: the pixel brightens as the control increases; blue: it darkens) and its intensity the magnitude. The two rows are on *individual* colour scales (the branch and select gradients differ in absolute magnitude by more than $10\times$), each with its own colour bar in absolute units; the per-panel number is the peak relative to that row's maximum. A pixel with no sensitivity maps to the same neutral light grey in both rows (the faint cannon outline), so the zero level is consistent. The gradient is largest at intermediate relaxation and vanishes toward the hard limit (large α , small T), consistent with Theorems 1 and 2.

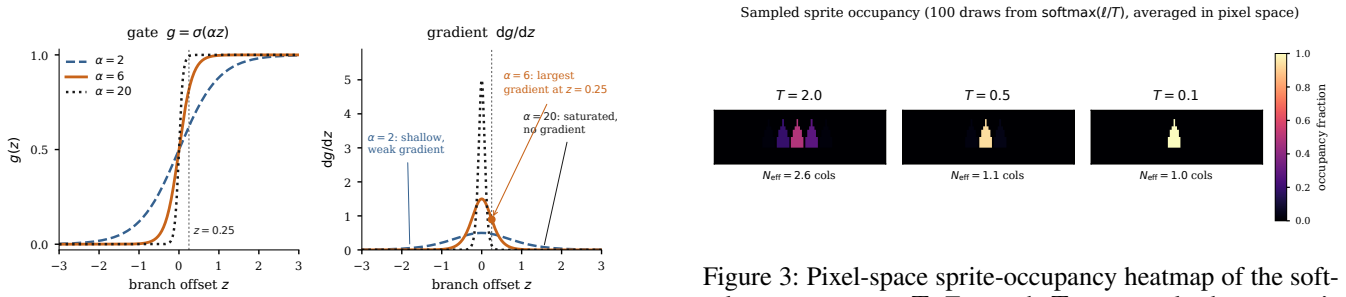


Figure 2: The soft-branch gate $g = \sigma(\alpha z)$ (left) and its gradient $dg/dz = \alpha \sigma(\alpha z)(1 - \sigma(\alpha z))$ (right) for $\alpha \in \{2, 6, 20\}$, with the operating point $z=0.25$ marked. Too large an α (here 20) saturates the gate, leaving essentially no gradient away from the switch; too small an α (here 2) gives a shallow gate whose gradient is weak and barely varies with z . The intermediate $\alpha=6$ places the operating point in the sigmoid's steep band and yields the largest, most localised gradient.

Figure 3: Pixel-space sprite-occupancy heatmap of the soft-select temperature T . For each T we sample the cannon's column 100 times from $w = \text{softmax}(\ell/T)$, render the hard cannon sprite at each sampled column, and average the 100 images; each pixel's value is the fraction of samples in which the sprite covered it—a Monte-Carlo estimate of the expected occupancy $\sum_k w_k \text{occ}_k$. At $T=2.0$ the draws spread over several columns (a blurred sprite of graded brightness, $N_{\text{eff}}=2.6$ effective columns); as T falls the occupancy collapses onto the target column ($N_{\text{eff}} \rightarrow 1$), a single sharp cannon at full occupancy.

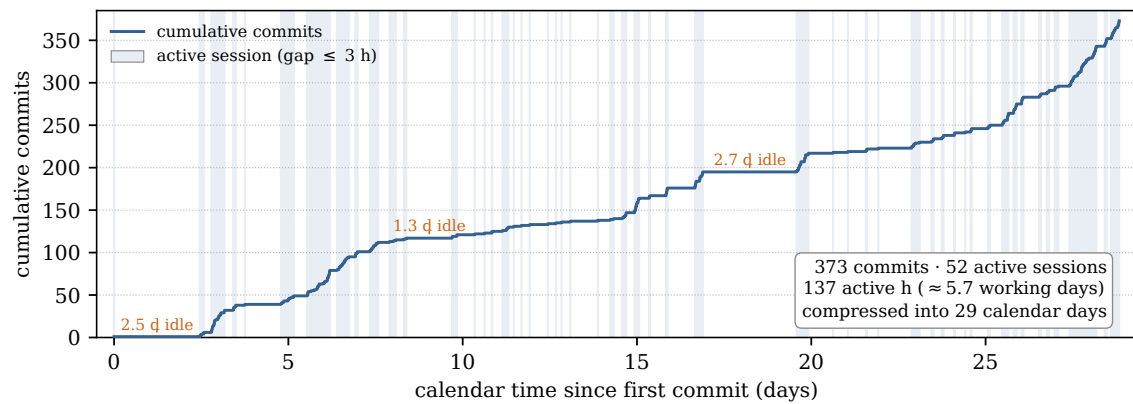


Figure 4: Implementation effort reconstructed from the version-control log. The step curve is the cumulative number of commits against calendar time; each shaded band is an active work session (a maximal run of consecutive commits no more than three hours apart), and the long flat stretches between bands are idle gaps, the largest of which are annotated. The 373 commits form 52 active sessions totalling ~ 137 hours of work—about 5.7 working days—spread across a 29-day calendar window.

References

- Goldberg, D. 1991. What Every Computer Scientist Should Know About Floating-Point Arithmetic. *ACM Computing Surveys*, 23(1): 5–48.
- IEEE. 2019. IEEE Standard for Floating-Point Arithmetic (IEEE Std 754-2019). IEEE Std 754-2019.